

DSA V2 — PHASE 0

JavaScript Foundation • September 7 → October 4, 2026

Mission: Build JavaScript ability for DSA while maximizing independent thinking. CodeWithHarry is the primary teaching resource; MDN is the reference; practice is driven by your own reasoning and homework.

Core rule: Do not advance because the calendar says so. Finish the current stage and pass its mastery gate first.

1. Learning System — Thinking First

Step	What you do
1. Learn	CodeWithHarry: learn only enough to understand the current concept.
2. Close	Close the lesson. Reproduce the idea from memory.
3. Think	For every problem: Input → Output → Example → constraints/edge cases → brute-force idea.
4. Implement	Write the solution from a blank JS file. No copying.
5. Debug	Use console/debugger first. Inspect your own assumptions.
6. Escalate	If still blocked: MDN → conceptual clarification → one hint → approach review.
7. Re-code	If you saw an approach, close it and implement it again from memory.
8. Explain	Explain the algorithm aloud and state time + auxiliary space.
9. Journal	Record the failure type: understanding, logic, coding, complexity, edge case.
10. Blind test	On Sunday, solve without topic labels or solution patterns.

2. Resources

Resource	Role	Rule
CodeWithHarry	Primary JavaScript learning resource	Learn the concept; do not type along blindly.
MDN Web Docs	JavaScript reference	Use when JS syntax/API behavior is unclear.
Your JS editor + console/debugger	Debugger debugging tool	Try to diagnose errors yourself before asking for help.
ChatGPT	Coach / hint system	Ask for concepts or one hint after a serious attempt; do not outsource thinking.

WEEK 1 — CORE JAVASCRIPT

September 7–13

■ Mon — Variables + types: let/const, primitives, typeof. Homework: age after 5 years, temperature conversion, rectangle/circle calculations, time conversions, swap variables.

■ Tue — Operators + conditions: arithmetic, comparison, ===, logical operators, ternary, if/else. Homework: sign, even/odd, largest/smallest, leap year, divisibility, grades, age eligibility, triangle, vowel/consonant, calculator.

■ Wed — Loops: for/while/do-while, break/continue. Homework: ranges, even/odd, sums, tables, factorial, reverse number, digit count/sum/product, numeric palindrome, prime, primes to n, factors, GCD, LCM.

■ Thu — Nested loops: square, rectangle, hollow rectangle, increasing/decreasing triangles, number patterns, pyramids, Floyd triangle, multiplication tables.

■ Fri — Functions + scope: declarations, parameters, arguments, return, arrow functions, default params, scope. Build a dd/subtract/multiply/divide/isEven/isPrime/factorial/reverseNumber/sumDigits/countDigits/isPalindrome/findMax/findMin/gcd/lcm/power/fibonacci.

■ Sat — Deep practice: solve 10 mixed Week-1 problems from blank files; explain Input/Output/Example/Approach/Edge cases.

■ Sun — Master test: 13 problems: largest of three, leap year, prime, GCD, LCM, reverse number, numeric palindrome, Fibonacci, factorial, sum digits, pyramid, calculator, reusable isPrime. Target: 10/13 independently.

WEEK 2 — ARRAYS + STRINGS

September 14–20

■ Mon — Arrays: creation, indexing, length, traversal, update, push/pop/shift/unshift. Solve maximum, minimum, sum, average, counts, search, target count, manual reverse, copy, replace/add/remove.

■ Tue — Mutation/copying: references, slice, splice, spread. Solve copy, insert/delete/replace by index, subarray, reverse without reverse(), duplicates, second-largest, sorted check, rotations.

■ Wed — Strings: indexing, length, case conversion, includes, startsWith, endsWith, slice, substring, split, join, trim. Solve reverse, character count, vowels, consonants, spaces, palindrome, remove spaces, occurrences, replace, reverse words, longest word.

■ Thu — Array methods: forEach, map, filter, reduce, find, findIndex, some, every, includes. Solve transformation/filtering/aggregation exercises.

■ Fri — Combined array + string: duplicates, second-largest, character/number frequency, intersection, union, common elements, move zeroes, merge, equality, rotation.

■ Sat — Deep practice: 10 blank-file problems: duplicates, second-largest, rotation, move zeroes, intersection, reverse string, palindrome, character frequency, longest word, number frequency.

■ Sun — Master test: 16 tasks across arrays, strings and map/filter/reduce. Target: 13/16 independently.

WEEK 3 — OBJECT + MAP + SET + MODERN JS

September 21–27

■ Mon — Objects: creation, access, update, delete, Object.keys/values/entries. Build student object and frequency counters.

■ Tue — Map: set/get/has/delete/size/iteration. Build student marks, frequency counter, most frequent element and grouping tasks. Explain Object vs Map.

■ Wed — Set: add/has/delete/size/iteration. Solve duplicate detection, unique extraction, intersections and duplicate-character tasks.

■ Thu — Destructuring/spread/rest: array/object destructuring, nested destructuring, swapping, copying/merging, rest parameters.

■ Fri — Data-structure choice: before coding, state Array vs Object vs Map vs Set and why. Solve word frequency, membership, grouping, duplicates, unique values, first repeated/unique character.

■ Sat — Deep practice: build Object frequency counter, Map frequency/grouping, Set duplicate detector/unique extractor; then solve 8 mixed problems.

■ Sun — Master test: 12 tasks. For every answer explain why the chosen data structure is appropriate.

WEEK 4 — SORTING + RECURSION + I/O + TESTING

September 28–October 4

■ Mon — Sorting: `sort()`, numeric comparator, ascending/descending, string length, object sorting. Solve 12 sorting exercises.

■ Tue — Recursion: base case, recursive case, call stack, parameters, return values. Solve countdown, count-up, factorial, sum, Fibonacci, power, reverse string, array sum/max, recursive binary search.

■ Wed — Coding-platform I/O: practice input → processing → output. Solve 10 small input/output exercises.

■ Thu — Clean functions + testing: refactor `isPrime`, `isPalindrome`, `findMax`, `removeDuplicates`, `frequencyMap`, `binarySearch`. Test normal, empty, single, duplicates and negative values.

■ Fri — Full revision: no new concepts. Randomly select earlier homework and solve from blank.

■ Sat — Final blind simulation: 10 unlabeled problems including second-largest, duplicate characters, frequency, in-place reverse, sorted check, first unique, binary search, recursive factorial, object sorting and duplicate detection.

■ Sun — Final Phase 0 gate: 25 mixed tests covering fundamentals, arrays, strings, `Object/Map/Set`, sorting and recursion. Pass only when you can start and explain small JS programs independently from a blank file.

3. Daily Thinking Template

Use this before writing code:

Problem: _____
Input: _____
Output: _____
Small example: _____
Edge cases: _____
Brute force idea: _____
Bottleneck: _____
My data structure/approach: _____
Why?: _____
Time: _____
Auxiliary space: _____
Mistake type: _____

4. Anti-Dependency Rules

- Do not immediately ask for a complete solution.
- Attempt for 20–30+ minutes when the problem is substantial.
- Use your console/debugger before asking for debugging help.
- Use MDN when the blocker is JavaScript behavior rather than algorithmic reasoning.
- If you see a solution, close it and re-code from memory.
- Keep some Sunday sessions completely AI-free.
- Re-solve missed problems on Day 1, 3, 7, 14 and 30.

5. Phase 0 Mastery Gate

You are ready to move on only when you can open a blank JS file and independently understand, plan, implement, test, debug and explain small programs using the Phase 0 topics. This matches the uploaded DSA V2 roadmap's mastery gate.

Next stage: Phase 1 — Logic Building. Do not start it merely because October 4 arrives; stay in Phase 0 if the gate is not passed.

6. GitHub V2 Philosophy

The repository template accompanying this PDF is designed so your GitHub records your thinking, not just final code. Each problem has a reasoning file, code file and result/mistake record. The goal is to make the repository evidence of independent problem solving.